

Bruno M. Banho

**QUANTIFLUX: METODOLOGIA DE ORQUESTRAÇÃO
DE ENXAMES DE AGENTES DE IA POR ONDAS E
PLANO DIRETOR DE ENGENHARIA**

Axio Engineering & Meta-Cognitive Systems
Laboratório de Arquitetura de Software e Agentes Autônomos

Orientador: Axio Engineering & Meta-Cognitive Systems

Brasil
2026

Resumo

Este documento apresenta o **Quantiflux**, um arcabouço metanalítico e formal para orquestração de múltiplos agentes autônomos de inteligência artificial em projetos de engenharia de software de alta complexidade. A metodologia fundamenta-se na separação estrita de quatro camadas de abstração (Conceitual, Estratégico, Tático e Operacional) e na execução orientada a Ondas (*Wave-based Execution*) derivadas de um Plano Diretor de Engenharia. Demonstra-se como o isolamento de contexto reduz drasticamente o *context bleed* e previne a degradação conceitual do sistema durante a codificação massiva por agentes de linguagem.

Palavras-chave: Orquestração de Agentes. Engenharia de Software. Plano Diretor. Execução por Ondas. Isolamento de Contexto.

Sumário

1	INTRODUÇÃO E FUNDAMENTAÇÃO	4
1.1	O Problema da Degradação Conceitual em Agentes Autônomos	4
1.2	A Filosofia Quantiflux	4
1.3	Objetivos da Metodologia	4
2	ARQUITETURA EM QUATRO CAMADAS	5
2.1	Estrutura de Abstração	5
2.1.1	Nível Conceitual	5
2.1.2	Nível Estratégico	5
2.1.3	Nível Tático	5
2.1.4	Nível Operacional	5
3	ORQUESTRAÇÃO POR ONDAS E PORTÕES DE QUALIDADE	6
3.1	Decomposição por Ondas (Wave-based Execution)	6
3.2	O Processo Quantiflux de Engenharia Meta-Orientada	6
3.2.1	Visão Geral do Processo	6
3.2.2	Detalhamento do Fluxo de Trabalho	6
3.2.2.1	Fase 1: Início e Modelagem Conceitual	6
3.2.2.2	Fase 2: Estruturação Estratégica e Decomposição	7
3.2.2.3	Fase 3: Isolamento Tático e Despacho	7
3.2.2.4	Fase 4: Execução Operacional e Portões de Onda	7
3.2.3	Detalhamento dos Subprocessos Integrados	7
3.2.3.1	Subprocesso: Análise e Mineração de Resíduos	7
3.2.3.2	Subprocesso: Validação Operacional no Portão	8
4	CONCLUSÃO E TRABALHOS FUTUROS	9
4.1	Considerações Finais	9
4.2	Evolução do Framework	9

1 Introdução e Fundamentação

1.1 O Problema da Degradação Conceitual em Agentes Autônomos

A utilização de modelos de linguagem de grande porte (LLMs) em tarefas de desenvolvimento de software trouxe capacidade de escrita de código em alta velocidade. Contudo, em projetos extensos, a atuação não estruturada de múltiplos agentes acarreta o fenômeno da *crise conceitual* ou *context drift*. Quando agentes especialistas atuam sem um plano unificado, decisões locais violam invariantes de arquitetura globais.

1.2 A Filosofia Quantiflux

O **Quantiflux** foi concebido para impor disciplina estrutural estrita antes da geração de qualquer linha de código. A premissa fundamental do framework é que a janela de contexto de um agente de nível estratégico é um ativo de alto valor. Para preservá-la, a execução de código é delegada a subagentes táticos temporários e isolados, operando sob o controle determinístico de um Plano Diretor de Engenharia (*Master Engineering Plan*).

1.3 Objetivos da Metodologia

- Formalizar a separação de responsabilidades em quatro níveis hierárquicos.
- Estabelecer a execução sequencial por Ondas (*Waves*) com portões de qualidade invioláveis.
- Garantir que mudanças estruturais drásticas sejam tratadas no nível de tese e plano diretor, e não como improvisos operacionais.

2 Arquitetura em Quatro Camadas

2.1 Estrutura de Abstração

A orquestração Quantiflux divide o fluxo de trabalho em quatro níveis distintos e complementares:

2.1.1 Nível Conceitual

Representa a tese e a modelagem formal do sistema. Define as entidades fundamentais, os contratos imutáveis e as invariantes de domínio. Não contém detalhes de implementação ou bibliotecas específicas.

2.1.2 Nível Estratégico

Converte a tese conceitual no Plano Diretor de Engenharia (*Master Engineering Plan*). Responsável por decompor o projeto em fases lógicas e ordenar as dependências entre módulos.

2.1.3 Nível Tático

Atua como o gerenciador de contexto e despachante. Filtra as informações relevantes do Plano Diretor para a Onda corrente e dispara subagentes especialistas via primitivas como `delegate_task`.

2.1.4 Nível Operacional

Constituído pelos operadores e executores de código. Trabalham em ambientes limpos, escrevendo testes unitários e implementações locais. Submetem seus artefatos para validação nos portões de qualidade.

3 Orquestração por Ondas e Portões de Qualidade

3.1 Decomposição por Ondas (Wave-based Execution)

Em vez de permitir que agentes modifiquem múltiplos componentes simultaneamente, o Quantiflux exige a ordenação em Ondas:

1. **Onda 0 (Core Contracts):** Definição de interfaces, DTOs e tipos abstratos.
2. **Onda 1 (Adapters & Persistência):** Implementação de adaptadores de infraestrutura e banco de dados.
3. **Onda 2 (Serviços & Lógica de Negócio):** Construção das regras de negócio consuming os contratos da Onda 0.
4. **Onda N (Interface & Integração Externa):** Camadas de exposição e APIs.

3.2 O Processo Quantiflux de Engenharia Meta-Orientada

3.2.1 Visão Geral do Processo

O processo operacional do Quantiflux descreve a sequência de engenharia necessária para guiar enxames de agentes em ondas de desenvolvimento seguras. Ele é estruturado em raias funcionais que delimitam as fronteiras de responsabilidade técnica entre os níveis de abstração: Conceitual, Estratégico, Tático e Operacional.

3.2.2 Detalhamento do Fluxo de Trabalho

3.2.2.1 Fase 1: Início e Modelagem Conceitual

- **Início do Fluxo:** O ciclo operacional se inicia no evento inicial localizado na raia Conceitual.
- **Formulação da Tese:** O engenheiro projeta a arquitetura mestre do sistema livre de detalhes operacionais de código.
- **Artefatos de Dados:** Gera-se o documento de dados **Invariantes e Contratos de Domínio** que guiará todas as camadas subsequentes.

3.2.2.2 Fase 2: Estruturação Estratégica e Decomposição

- **Elaboração do Master Engineering Plan:** Na raia Estratégica, a tese conceitual é convertida no documento prático localizado em `.hermes/plans/master_plan.md`.
- **Decomposição por Ondas:** O plano é fracionado em blocos incrementais e independentes denominados Ondas (Waves).
- **Análise de Resíduos (Feedback Loop):** Uma etapa dedicada analisa commits, logs e históricos anteriores (resíduo semântico) para refinar o plano estrategicamente antes de enviar tarefas para a fase operacional (Cherry-Pick / Feedback Loop).

3.2.2.3 Fase 3: Isolamento Tático e Despacho

- **Filtragem de Contexto:** Na raia Tática, a informação é filtrada para conter estritamente as metas da Onda ativa, prevenindo o fenômeno de *context bleed*.
- **Disparo de Tarefas:** Subagentes especializados são despachados dinamicamente via primitivas de controle (`delegate_task`), recebendo exclusivamente o manifesto isolado da Onda.

3.2.2.4 Fase 4: Execução Operacional e Portões de Onda

- **Implementação por Operadores:** Os operadores em nível operacional executam a codificação sob a estrita disciplina de TDD.
- **Verificação e Portões de Onda (Wave Boundary Gates):** O fluxo alcança o portão de verificação de testes unitários e linter. Se aprovado, o estado do repositório é consolidado (`git status limpo`).
- **Compilação de Artefatos (CI/CD):** Dispara-se o pipeline automatizado via GitHub Actions para validação e compilação das especificações LaTeX para geração do manual PDF.

3.2.3 Detalhamento dos Subprocessos Integrados

3.2.3.1 Subprocesso: Análise e Mineração de Resíduos

Este subprocesso estratégico é acionado iterativamente para garantir que lições e alterações de código residuais sejam integradas sem introduzir desvios na tese conceitual:

1. **Capturar Histórico:** Coleta logs de commits e buffers de conversas anteriores.
2. **Executar Cherry-Pick:** Filtra e seleciona commits de mudanças legítimas para reintegração na tese.

3. **Validar contra Invariantes:** Verifica se os trechos recuperados colidem com as definições conceituais do domínio.

3.2.3.2 Subprocesso: Validação Operacional no Portão

Subprocesso estrito que bloqueia a promoção de ondas se houver instabilidade lógica:

1. **Executar Testes Unitários:** Dispara a suíte local de testes automatizados.
2. **Análise Estática:** Valida estilos e assinaturas contra os contratos da Onda 0.
3. **Garantia de Estado:** Garante que nenhum arquivo não-rastreado polua o repositório antes do fechamento.

4 Conclusão e Trabalhos Futuros

4.1 Considerações Finais

O Quantiflux estabelece um padrão rigoroso de engenharia para o desenvolvimento assistido por exames de agentes de IA. Ao tratar a estratégia de software como um processo determinístico alimentado por planos diretores e validado por portões de borda de onda, mitiga-se o risco de colapso arquitetural em projetos complexos.

4.2 Evolução do Framework

Como trabalhos futuros, prevê-se a integração nativa dos portões de onda a pipelines de CI/CD automatizados via GitHub Actions e o aprimoramento dos algoritmos de compressão de contexto para subagentes táticos.